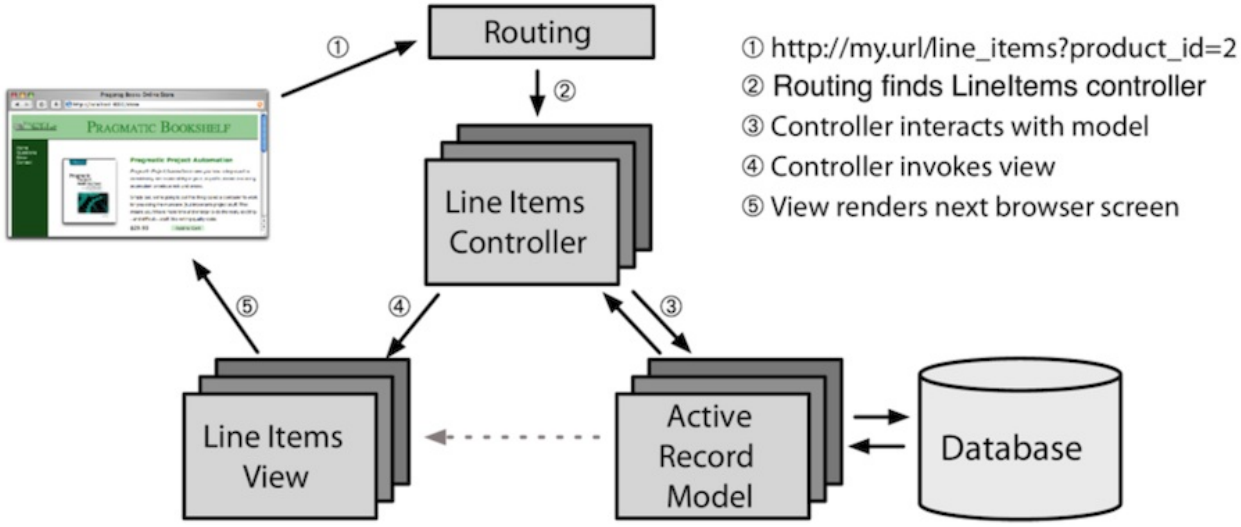
Rails handles an incoming request as shown in the following figure. In this example, the application has previously displayed a product catalog page, and the user has just clicked the Add to Cart button next to one of the products. This button posts to http://localhost:3000/line_items?product_id=2, where `line_items` is a resource in the application and `2` is the internal ID for the selected product.



The routing component receives the incoming request and immediately picks it apart. The request contains a path (`/line_items?product_id=2`) and a method (this button does a `POST` operation; other common methods are `GET`, `PUT`, `PATCH`, and `DELETE`). In this simple case, Rails takes the first part of the path, `line_items`, as the name of the controller and the `product_id` as the ID of a product. By convention, `POST` methods are associated with `create` actions. As a result of all this analysis, the router knows it has to invoke the `create` method in the `LineItemsController` controller class (we'll talk about naming conventions in *Naming Conventions*).

The `create` method handles user requests. In this case, it finds the current user's shopping cart (which is an object managed by the model). It also asks the model to find the information for product 2. It then tells the shopping cart to add that product to itself. (See how the model is being used to keep track of all the business data? The controller tells it *what* to do, and the model knows *how* to do it.)

Now that the cart includes the new product, we can show it to the user. The controller invokes the view code, but before it does, it arranges things so that the